

D598 Task 2: Coding

1. Import the data file into a data frame.

```
#Import the Excel file as a data frame
df_input = pd.read_excel(r"C:\Users\PaceW\Desktop\WGU\Courses\D598 - Analytics Programming\Task 2 Coding\D598 Data Set.xlsx")
```

2. Identify any duplicate rows in the data set.

```
#Identify duplicate records, provide count, Business ID for later triage, then remove the duplicates
df_duplicates = df_input[df_input.duplicated()]
duplicates_count = len(df_duplicates)
print(f'Number of duplicate records: {duplicates_count}')

#Provide the Business ID for the duplicate record for easy review.
if len(df_duplicates) > 0:
    print(df_duplicates[['Business ID']])
    df_input = df_input.drop_duplicates()
    print(f'{duplicates_count} duplicate records removed\n')
else:
    print('No duplicates were found in the file.\n')
```

```
Number of duplicate records: 0
No duplicates were found in the file.
```

Using a modified input file, I confirmed my duplicate handling.

```
Number of duplicate records: 3
  Business ID
4    197452013
7    252122013
9    293322013
3 duplicate records removed
```

- Group all IDs by state, then run descriptive statistics (mean, median, min, & max) for all numeric variables by state and store this result as a new data frame. (Code should be modified from “D598 Task 2 Original Code” in the supporting documents section)

```
#Group all rows by Business State
df_state_cols = [ 'Total Long-term Debt', 'Total Equity', 'Debt to Equity',
                  'Total Liabilities', 'Total Revenue', 'Profit Margin']
df_state_stats = df_input.groupby('Business State')

#Descriptive statistics (mean, median, min, and max)
#for Total Long-term Debt, Total Equity, Debt to Equity,
#Total Liabilities, Total Revenue, Profit Margin
desc_stats = ['mean', 'median', 'min', 'max']
df_desc_stats = df_state_stats[df_state_cols].agg(desc_stats)
df_desc_stats.columns = ['_'.join([col, stat]) for col, stat in df_desc_stats.columns]
print('Descriptive Statistics by State:')
display(df_desc_stats)
```

Sample output of the descriptive statistics.

Descriptive Statistics by State:

	Total Long-term Debt_mean	Total Long-term Debt_median	Total Long-term Debt_min	Total Long-term Debt_max	Total Equity_mean	Total Equity_median	Total Equity_min	Total Equity_max	Debt to Equity_mean	Debt to Equity_median
Business State										
Alabama	6.743890e+08	6.743890e+08	5.314000e+06	1.343464e+09	1.441039e+09	1.441039e+09	2.405848e+07	2.858019e+09	0.345473	0.345473
Arizona	6.508800e+07	6.508800e+07	6.508800e+07	6.508800e+07	5.915300e+07	5.915300e+07	5.915300e+07	5.915300e+07	1.100333	1.100333
Arkansas	8.960450e+07	8.960450e+07	7.036600e+07	1.088430e+08	1.082420e+08	1.082420e+08	1.005380e+08	1.159460e+08	0.844746	0.844746
California	3.273964e+07	4.721500e+06	1.500000e+04	3.349970e+08	4.535982e+07	1.427150e+07	-1.569100e+07	3.211750e+08	0.169920	0.154170
Colorado	8.470025e+07	2.932700e+07	3.375000e+06	3.753220e+08	2.007589e+08	6.077850e+07	1.284525e+07	9.339710e+08	0.619971	0.874711
Connecticut	1.851200e+07	1.851200e+07	1.851200e+07	1.851200e+07	4.399091e+06	4.399091e+06	4.399091e+06	4.399091e+06	4.208142	4.208142

4. Filter the data frame to identify all businesses with debt-to-equity ratios that are negative.

```
#Filter df_desc_stats to identify all businesses with debt-to-equity (dbteq) ratios that are negative
df_negative_dbteq = df_input[df_input['Debt to Equity'] < 0].sort_values(by = 'Debt to Equity')
print('Businesses With A Negative Debt To Equity Ratio')
df_negative_dbteq.reset_index(drop=True, inplace=True)
display(df_negative_dbteq[['Business ID', 'Business State', 'Debt to Equity']])
print(f'There are {len(df_negative_dbteq)} businesses with a negative Debt to Equity Ratio\n')
```

Results:

Businesses With A Negative Debt To Equity Ratio			
	Business ID	Business State	Debt to Equity
0	14535932013	Montana	-4.283552
1	934562013	Ohio	-2.370953
2	9323722013	California	-1.374036
3	8343652013	Washington	-0.798921
4	10919832013	Utah	-0.557949
5	11245242013	California	-0.269484
6	14639722013	New York	-0.014343

There are 7 businesses with a negative Debt to Equity Ratio

5. Create a new data frame that provides the debt-to-income ratio for every business in the data set. Debt-to-income ratio is defined as long-term debt divided by revenue.

```
#Find the Debt-to-income ratio for every business and store in a new data frame.
#Debt-to-income ratio = Long-term debt / revenue

#Find businesses with a Total Revenue = $0
df_zero_revenue = df_input[df_input['Total Revenue'] == 0]

#Warn the user if records are found with $0 Total Revenue. Provide Business ID for triage.
if not df_zero_revenue.empty:
    print('WARNING: The following businesses have a Total Revenue = $0 and will be skipped:')
    display(df_zero_revenue[['Business ID']])

#Filter records out
df_filtered_rev = df_input[df_input['Total Revenue'] != 0]
df_input = df_filtered_rev.copy()

#Calculate Debt-to-Income Ratio
df_debt_income_ratio = df_filtered_rev[['Business ID', 'Total Long-term Debt', 'Total Revenue']].copy()
df_debt_income_ratio['Debt-to-Income Ratio'] = df_debt_income_ratio['Total Long-term Debt'] / df_debt_income_ratio['Total Revenue']
df_debt_income_ratio = df_debt_income_ratio.sort_values(by = 'Debt-to-Income Ratio', ascending=False)
print('Debt-to-income ratio for each business')
df_debt_income_ratio.reset_index(drop=True, inplace=True)
display(df_debt_income_ratio)
```

Debt-to-income ratio for each business

	Business ID	Total Long-term Debt	Total Revenue	Debt-to-Income Ratio
0	13752052013	3.400000e+07	7616000	4.464286
1	12826482013	3.183823e+09	999506000	3.185397
2	3518172013	1.142368e+09	584401000	1.954767
3	3158522013	3.140516e+09	1832253000	1.714019
4	15456542013	6.055000e+08	365200000	1.657996
...	...	...	...	...
145	1021092013	2.500000e+04	15383877	0.001625
146	7889202013	1.500000e+04	12249000	0.001225

Once again using a modified test file I was able to confirm the code will handle records with \$0.00 Total Revenue.

WARNING: The following businesses have a Total Revenue = \$0 and will be skipped:

Business ID

0 41872013

1 76232013

2 160992013

3 197452013

5 241042013

Debt-to-income ratio for each business

- Concatenate the debt-to-income ratio data frame you created with the original data frame.

```
#Concatenate the debt-to-income ratio data frame with the original data frame
df_final = pd.merge(df_input, df_debt_income_ratio[['Business ID', 'Debt-to-Income Ratio']], on='Business ID', how='left')
print('Final Output')
df_final.reset_index(drop=True, inplace=True)
display(df_final)
```

Final Output

	Business ID	Business State	Total Long-term Debt	Total Equity	Debt to Equity	Total Liabilities	Total Revenue	Profit Margin	Debt-to-Income Ratio
0	41872013	Kentucky	16889000.0	1.804600e+07	0.935886	2.598600e+07	136753000	0.023663	0.123500
1	76232013	Iowa	6252000.0	1.829362e+07	0.341758	1.447400e+07	34226553	0.265015	0.182665
2	160992013	Texas	19200000.0	1.778580e+08	0.107951	7.278700e+07	384196000	0.130413	0.049974
3	197452013	Delaware	117592000.0	2.787730e+08	0.421820	5.587490e+08	444306000	0.196768	0.264664
4	241042013	Illinois	4408000.0	5.206400e+07	0.084665	1.989800e+07	121541000	0.168305	0.036268
...	...	...	...	...	...	...	...	...	...
145	14733342013	California	6602000.0	3.872871e+07	0.170468	2.183800e+07	78356493	0.213007	0.084256
146	14871012013	Maryland	56000000.0	2.813310e+08	0.199054	1.476180e+08	298732000	0.353230	0.187459

References

Pandas documentation#. pandas documentation - pandas 2.3.1 documentation. (n.d.).

<https://pandas.pydata.org/docs/index.html>

WGU Analytics Programming course materials, July 2025